

Herencias e Interfaces.

Laboratorio 3

Oscar Lasso

Juan Diego Gaitán

Profesor:

Maria Irma Diaz Roza

Desarrollo Orientado a Programación de Objetos

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá D.C.

PARTE I. Conociendo el proyecto

A Revisando línea base [En [lab03.doc](#)]

1. En el directorio descarguen los archivos contenidos de [forest.zip](#). Revisen el código: (a) ¿Cuántos paquetes tiene? (b) ¿Cuál es el propósito del paquete presentación? (c) ¿Cuál es el propósito del paquete dominio?
2. Revisen el paquete de dominio, (a) ¿Cuáles son los diferentes tipos de componentes de este paquete? (b) ¿Qué implica cada uno de estos tipos de componentes?
3. Revisen el paquete de presentación, (a) ¿Cuántos componentes tiene? (b) ¿Cuántos métodos públicos propios (no heredados) ofrece?
4. Para ejecutar un programa en java, (a) ¿Qué método se debe ejecutar? (b) ¿En qué clase se encuentra?
5. Ejecuten el programa. (a) ¿Qué funcionalidades ofrece? (b) ¿Qué hace actualmente? (c) ¿Por qué?

Deben ejecutar la aplicación java, no crear un objeto como lo veníamos haciendo

- 1)
 - a) Tiene 2 paquetes: presentation y domain
 - b) Contiene la interfaz GUI del programa Forest
 - c) Contiene las clases principales del funcionamiento y lógica
- 2)
 - a) Tiene una clase Tree que hereda de la clase abstracta Living Thing e implementa la clase interfaz Thing. Además contiene una clase concreta Forest.

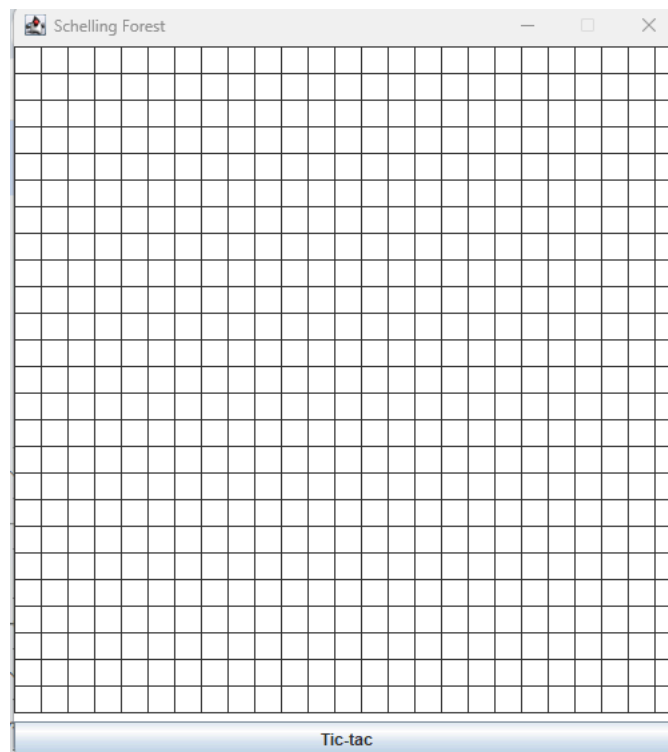
Clase concreta:
Clase que puede instanciarse (crear objetos) porque tiene implementados todos sus métodos.

Clase abstracta:
Clase que no puede instanciarse directamente y sirve como base para otras clases, pudiendo tener métodos abstractos (sin implementación) y métodos implementados.

Herencia:
Mecanismo de la programación orientada a objetos mediante el cual una clase adquiere los atributos y métodos de otra clase usando extends.

Interfaz:
Tipo que define un conjunto de métodos que una clase debe implementar, actuando como un contrato de comportamiento, usando implements.
 - b)

- 3)
- a) Tiene dos clases: Forest GUI que hereda de JFrame y PhotoForest que extiende de JPanel. Estas juntas representan la interfaz gráfica del programa
 - b) 6
- 4)
- a) Se debe ejecutar el método main de la clase que contiene la interfaz del programa.
 - b) Se encuentra en el paquete: presentation



- 5)
- a) Ofrece el botón Tic-Tac y el panel con un grid.
 - b) Actualmente sólo ofrece el grid visual.
 - c) Porque no se han implementado ninguna otra clase, ni implementado la funcionalidad del botón Tic-Tac

B Realizando ingeniería inversa. Arquitectura General [En [lab03.doc](#) [forest.astah](#)]

1. Consulten el significado de las palabras `package` e `import` de java. (a) ¿Qué es un paquete? (b) ¿Para qué sirve? (c) ¿Para qué se importa? (d) Expliquen su uso en este programa.
2. Revisen el contenido del directorio de trabajo y sus subdirectorios. Describan su contenido. (a) ¿Qué coincidencia hay entre paquetes y directorios?
3. Adicionen al diseño la arquitectura general con un diagrama de paquetes en el que se presente los paquetes y las relaciones entre ellos. Consulte la referencia en Moodle.
En astah, crear un diagrama de clases (cambiar el nombre por Package Diagram0)

1) a) ¿Qué es un paquete?

Un paquete es un grupo de clases e interfaces relacionadas que se organizan dentro de un mismo espacio de nombres en Java, similar a una carpeta.

b) ¿Para qué sirve?

Sirve para organizar el código, evitar conflictos de nombres y separar partes del programa según su función.

c) ¿Para qué se importa?

La palabra `import` se usa para permitir que una clase utilice clases que están en otro paquete.

d) ¿Cómo se usa en este programa?

El programa tiene dos paquetes:

`presentation`: contiene la interfaz gráfica.

`domain`: contiene la lógica del simulador (`Tree`, `Forest`, etc.).

El paquete `presentation` importa clases de `domain` para poder usar la lógica del modelo del bosque.

2)

El directorio de trabajo contiene varias carpetas:

UML: contiene los diagramas del sistema.

Código fuente: contiene el programa.

Dentro de código fuente se encuentra una documentación que contiene archivos explicativos del proyecto y un package que contiene dos carpetas principales:

domain: contiene las clases que implementan la lógica del programa.

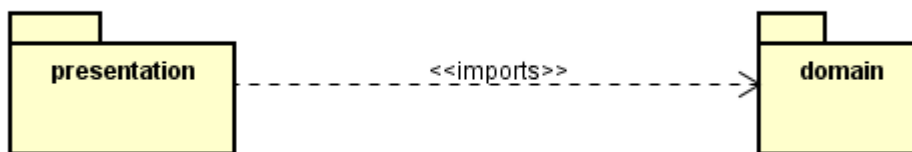
presentation: contiene las clases relacionadas con la interfaz gráfica.
Cada una de estas carpetas incluyen sus clases y el archivo de definición del paquete.

(a) ¿Qué coincidencia hay entre paquetes y directorios?

La coincidencia es que cada paquete en Java corresponde a un directorio en el sistema de archivos.

Es decir, la estructura de paquetes del programa coincide con la estructura de carpetas del proyecto.

3)



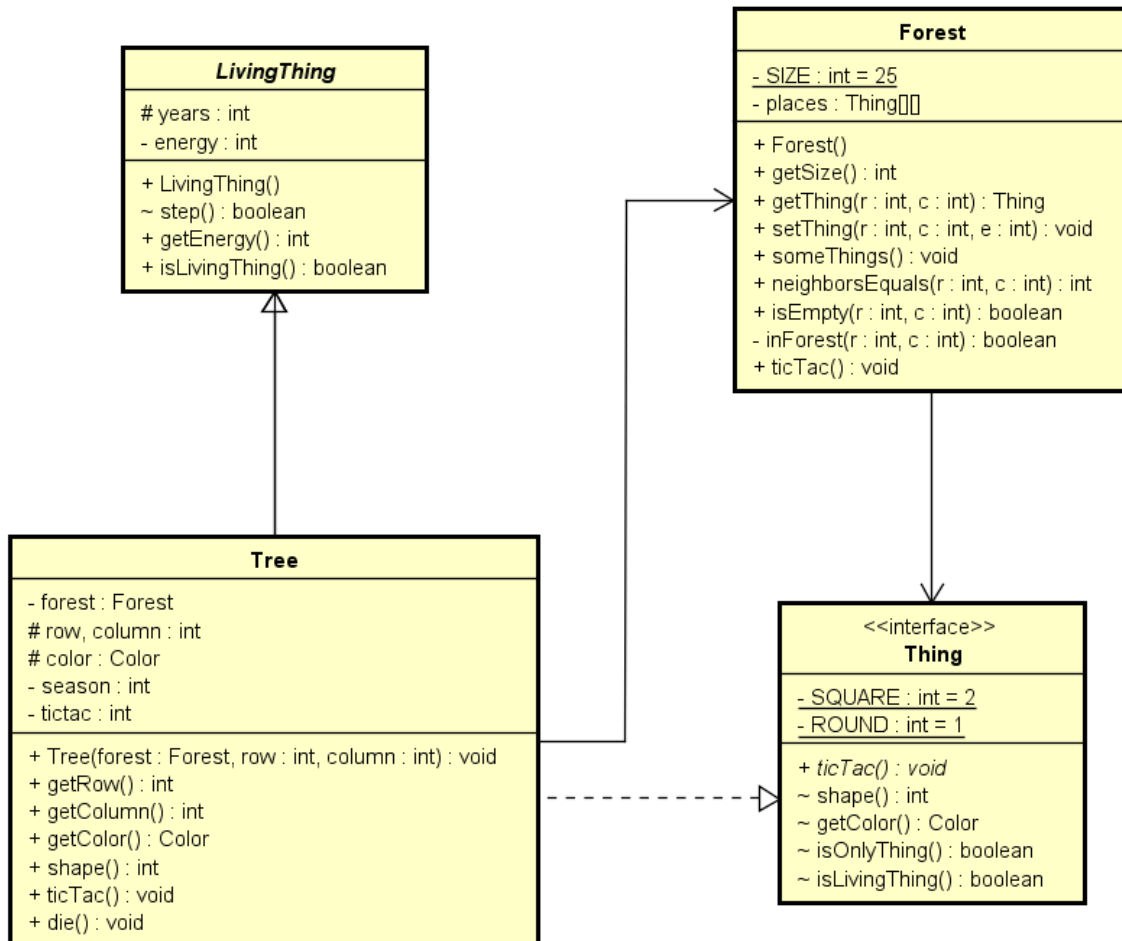
C Realizando ingeniería inversa. Arquitectura detallada [En [lab03.doc](#) [forest.astah](#)]

1. Para preparar el proyecto para **MDD**. Completen el diseño detallado del paquete de dominio. (a) ¿Qué componentes hacían falta?
2. Completen el diseño detallado del paquete de presentación. (a) ¿Por qué hay dos clases y un archivo .java?
3. Adicione la clase de pruebas necesaria para **BDD** en un paquete independiente de test. (No lo adicione al diagrama de clases) (a) ¿Qué clase debe importar? (b) ¿Por qué?

1)

a)

pkg



2)

a) Porque en Java es posible declarar múltiples clases en un mismo archivo .java, siempre que solo una sea public. En ForestGUI.java conviven:

3)

a) Debe importar:

- `domain.Forest` — la clase principal bajo prueba
- `domain.Tree` — para verificar instancias
- `domain.Thing` — para acceder a las constantes `ROUND` y `SQUARE`

- domain.LivingThing — para verificar getEnergy() y years
- org.junit.Assert.* — para los métodos assertEquals, assertNotNull, etc.
- org.junit.Before — para el método de inicialización
- org.junit.Test — para marcar los métodos de prueba

b)

Porque el paquete test es independiente de domain, no tiene visibilidad automática de sus clases. En Java cada paquete es un espacio de nombres separado, por lo que para usar Forest desde test es obligatorio importarla explícitamente. Sin el import, el compilador no sabe a qué Forest se hace referencia.

Las importaciones de JUnit (Assert, Before, Test) son necesarias porque las anotaciones y métodos de prueba están definidos en la librería JUnit, que es externa al código del proyecto.

PARTE II. Desarrollando el proyecto

Para desarrollar esta aplicación vamos a considerar algunos ciclos.

A Ciclo 1. Iniciando con árboles [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

1. Estudie la clase `Forest` . (a) ¿Qué tipo de colección usa para albergar cosas? (b) ¿Puede tener árboles ? (c) ¿Por qué?
2. Estudie el código asociado a la clase `Tree` , (a) ¿de qué color es? (b) ¿qué forma usa para pintarse? (c) ¿cuándo cambia de color ? (d) ¿qué componentes definen la clase `Tree` ? Justifique sus respuestas.
3. `Tree` por ser un `LivingThing` , (a) ¿qué atributos tiene? (b) ¿qué puede hacer (métodos)? (c) ¿qué decide hacer distinto (sobre-escritura)? (d) ¿qué no puede hacer distinto (métodos finales)? (e) ¿qué debe aprender a hacer (métodos abstractos)? Justifique sus respuestas.
4. Por comportarse como un `Thing` , (a) ¿qué sabe hacer? (b) ¿qué decide hacer distinto? (c) ¿qué no puede hacer distinto? (d) ¿qué debe aprender a hacer? Justifique sus respuestas.
5. De acuerdo a lo anterior un `Tree` , (a) ¿Cómo actúa?
6. Ahora vamos a crear dos árboles en diferentes posiciones (10,10) (15,15) , llámelos `beard` y `soul` , usando el método `something` s. Ejecuten el programa, (a) ¿Qué pasa con los árboles ? (b) ¿Por qué? (c) Capturen una pantalla significativa.
7. Diseñen, construyan y prueben el método llamado `ticTac()` de la clase `Forest` .
8. (a) ¿Cómo quedarían `beard` y `soul` después de 4,7,10 y 12 tic-tac? Ejecuten el programa. Capturen pantallas significativas en los momentos correspondientes. (b) ¿Es correcto?

1)

- a) La clase Forest usa un arreglo bidimensional de tipo Thing:
 - private Thing [][] places;
- b) Sí, el bosque puede contener árboles (Tree).
- c) Porque la clase Tree implementa la interfaz Thing:
 - public class Tree extends LivingThing implements Thing { ... }

La matriz places es de tipo `Thing[ ][ ]`, por lo que puede almacenar cualquier objeto cuya clase implemente dicha interfaz. Gracias al polimorfismo, una referencia de tipo `Thing` puede apuntar a un objeto `Tree`.

2)

a) `Tree` no tiene un único color fijo, cambia cíclicamente entre 4 colores según el valor de `tictac % 4`, pero inicialmente es rosado

b) El método `shape()` retorna la constante `Thing.ROUND`:

```
- public final int shape() {
    return Thing.ROUND;
}
```

Por tanto, usa forma redonda

c) El color cambia en cada llamada a `ticTac()`, que incrementa el contador `tictac` y recalcula el color. El ciclo completo dura 4 tics

d) `Tree` se define por la combinación de herencia + implementación de interfaz + atributos propios

3)

a) `Tree` hereda dos atributos de `LivingThing`:

```
- protected int years;
private int energy;
```

Y agrega los suyos propios:

```
- private Forest forest;
protected int row, column;
protected Color color;
private int season;
private int tictac;
```

b) Métodos heredados:

```
- Consumir energía (step)
Consultar energía
Saber si es ser vivo
```

Métodos propios:

```
- Tree puede ejecutar comportamientos heredados como consumir
energía y además tiene comportamientos propios como cambiar de
estado con ticTac o morir.
```

c) `LivingThing` no declara métodos abstractos propios, pero `Tree` sobrescribe métodos default de la interfaz `Thing`:

```
- public final int shape() {
    return Thing.ROUND;    // sobrescribe default SQUARE
}
```



```
public final Color getColor() {
    return color;      // sobrescribe default Color.black
}
```

- d) Los tres métodos heredados de LivingThing son final, Tree no puede sobrescribirlos bajo ninguna circunstancia
- e) LivingThing es una clase abstracta pero no declara métodos abstractos propios. Sin embargo, Tree debe implementar los métodos de la interfaz Thing que no tienen implementación default

4)

- a) Por implementar Thing, Tree hereda automáticamente los métodos con implementación default:
 - default int shape()
 - default Color getColor()
 - default boolean isOnlyThing()
 - default boolean isLivingThing()

Estos métodos están disponibles en Tree sin necesidad de escribir nada, a menos que decida sobrescribirlos.

- b) Puede sobrescribir (override) métodos default de la interfaz
 - c) No puede evitar implementar métodos obligatorios
 - d) Debe implementar los métodos abstractos de la interfaz
- En este caso: - public void ticTac();

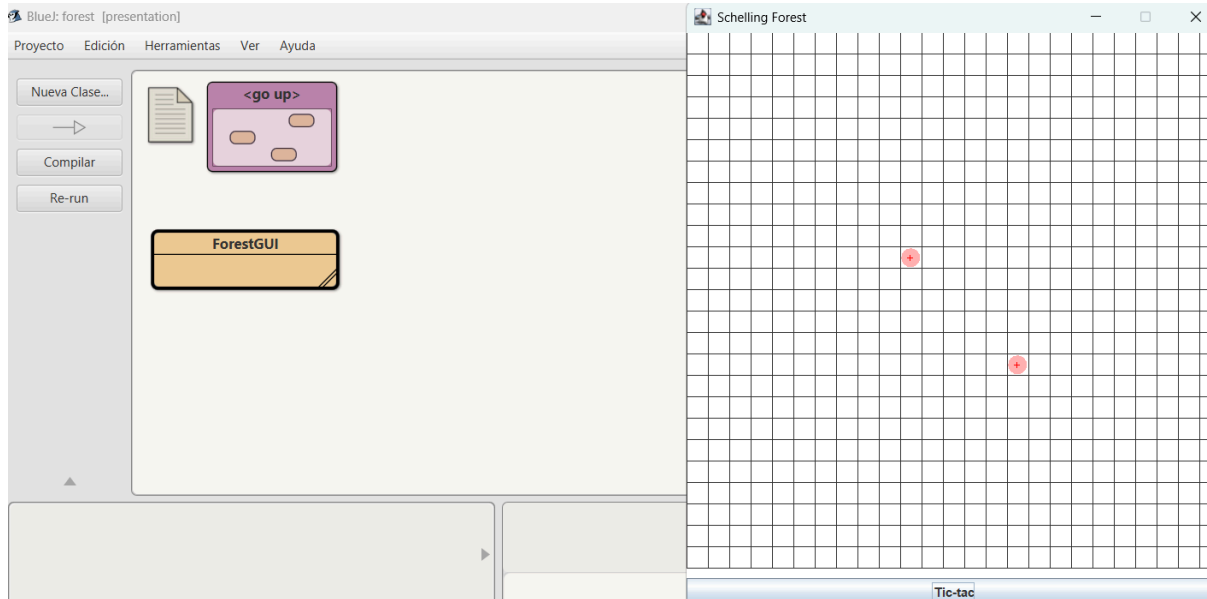
5)

- a) Un Tree actúa como la composición de tres contratos simultáneos: su ciclo de vida como LivingThing, su comportamiento visual como Thing, y su lógica propia de posición y estaciones. El comportamiento de un Tree es la suma integrada de sus tres contratos actuando en cada tic.

6)

- a) Al ejecutar el programa se observa lo siguiente:
 - Los dos árboles aparecen visibles en la cuadrícula como óvalos de color rosa (PINK) en las posiciones (10,10) y (15,15).
 - Cada uno muestra el símbolo + en rojo encima, indicando que su energía es ≥ 50 (inician con energy = 100).
 - Al presionar Tic-tac ambos avanzan de estación simultáneamente: PINK → GREEN → ORANGE → GRAY → PINK...

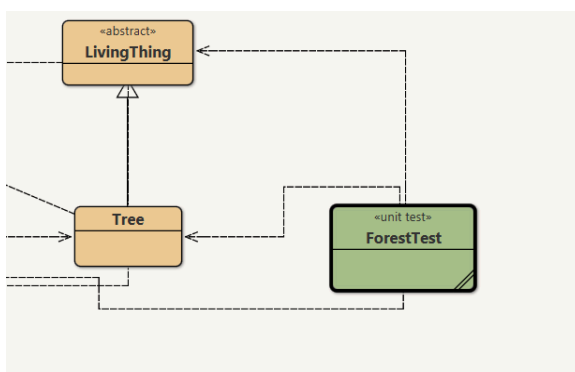
- Al llegar al invierno ($\text{tictac} \% 4 == 3$) cada árbol consume 1 de energía via `step()`.
- Después de 100 inviernos (400 tics) su energía llega a 0, `step()` retorna false, se llama `die()` y desaparecen de la cuadrícula.
- Porque la cadena de responsabilidades funciona exactamente como fue diseñada



7)

```
public void ticTac(){
    for (int r = 0; r < SIZE; r++) {
        for (int c = 0; c < SIZE; c++) {
            if (places[r][c] != null) {
                places[r][c].ticTac();
            }
        }
    }
}
```

8)



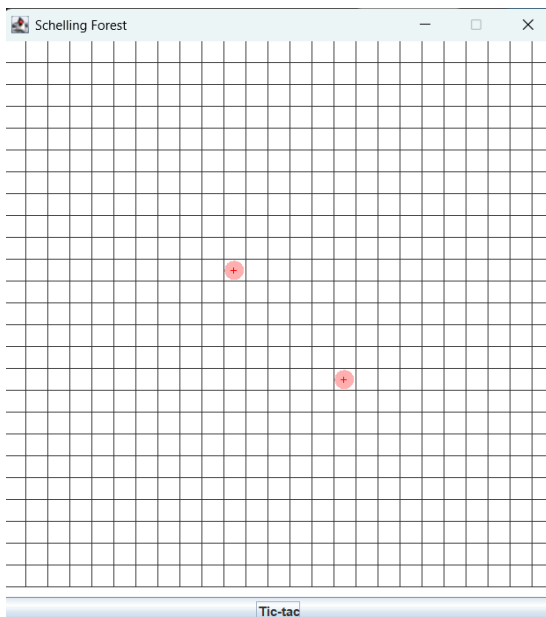
```
@Test
public void testTicTacNullSafe() {
    try {
        forest.ticTac();
    } catch (NullPointerException e) {
        fail("ticTac() no debe fallar en celdas null");
    }
}

@Test
public void testUnTicCambiaAGreen() {
    forest.ticTac();
    assertEquals(Color.GREEN, forest.getThing(10, 10).getColor());
    assertEquals(Color.GREEN, forest.getThing(15, 15).getColor());
}

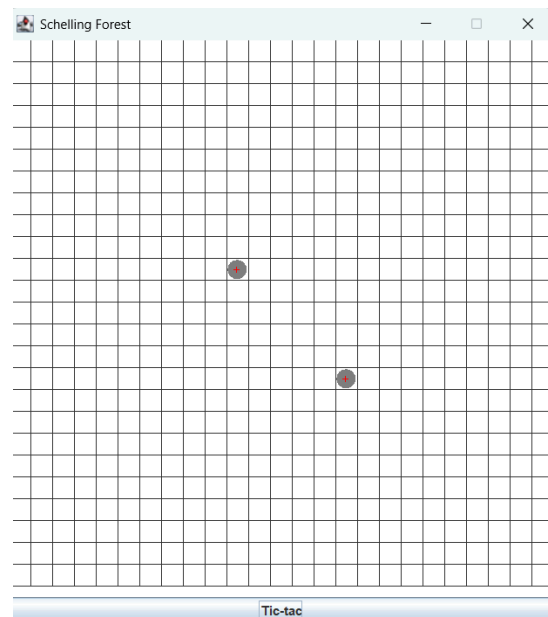
@Test
public void testDosTicsCambiaAOrange() {
    forest.ticTac();
    forest.ticTac();
    assertEquals(Color.ORANGE, forest.getThing(10, 10).getColor());
    assertEquals(Color.ORANGE, forest.getThing(15, 15).getColor());
}

@Test
public void testTresTicsCambiaAGrayYConsumeEnergia() {
    forest.ticTac();
    forest.ticTac();
    forest.ticTac();
}
```

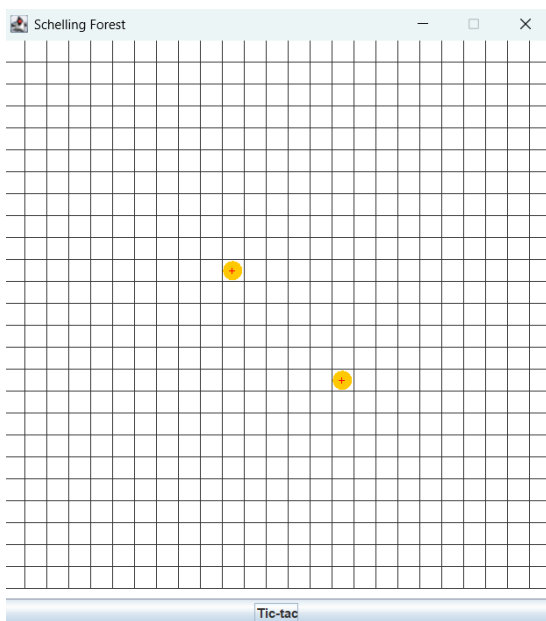
4



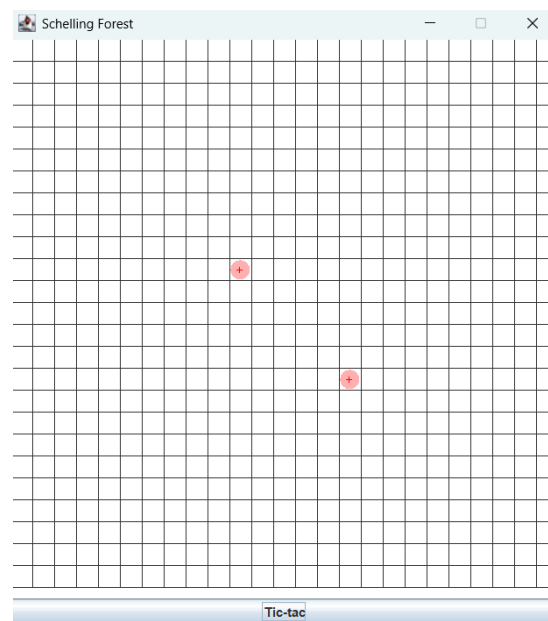
7



10



12



b) el comportamiento es correcto porque respeta exactamente el código implementado.

B Ciclo 2. Incluyendo ardillas [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir tener ardillas en el bosque. Ellas (i) son cafés y a medida que pasan los años se van tornando amarillentas, (ii) después de 10 años mueren, (iii) si quedan dos vecinas con una celda intermedia vacía, allí nace una nueva ardilla, (iv) se mueven aleatoriamente por el bosque

1. Para implementar esta nueva `LivingThing` (a) ¿cuáles métodos se sobre-escriben (overriding)?
2. Diseñen, construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)
3. Adicione una pareja de ardillas, de nombre `scrat` y `sandy`, (a) ¿Cómo quedarían después de 5 Tic-tac? Ejecuten el programa y hagan 5 clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?

1)

a) Comparando con `Tree`, `Squirrel` también extiende `LivingThing` e implementa `Thing`, pero necesita sobre-escribir comportamientos distintos:

- `ticTac()`: lógica propia: moverse, reproducirse, envejecer, morir a los 10 años
- `getColor()`: inicia café, se torna amarillenta con los años
- `shape()`: devuelve `SQUARE` — forma de ardilla

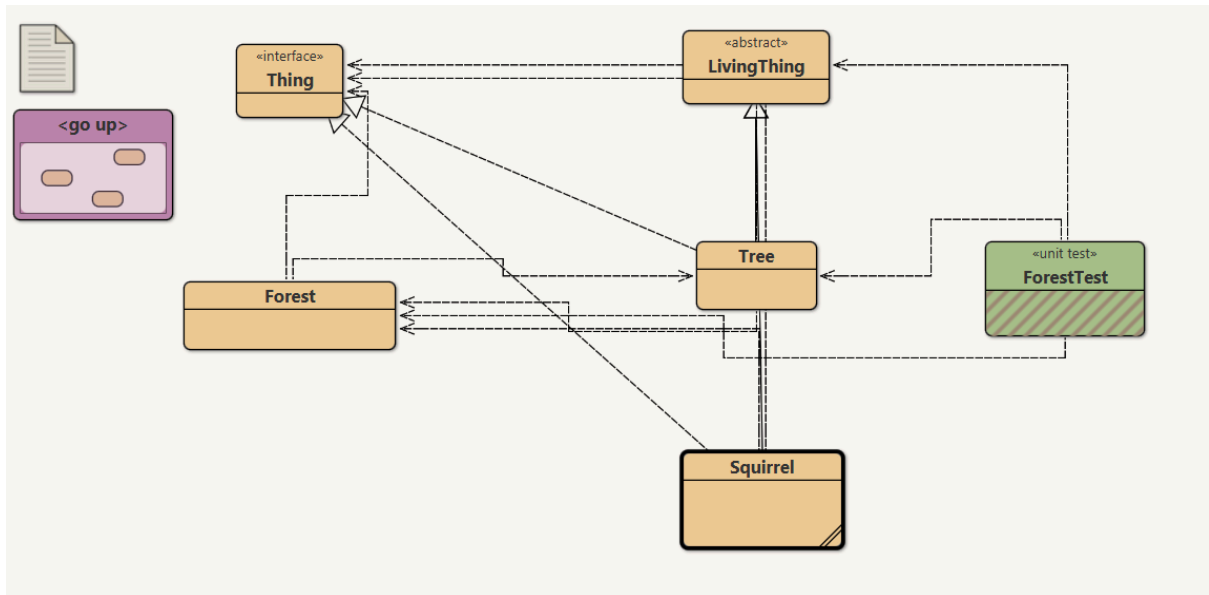
Los métodos `step()`, `getEnergy()` e `isLivingThing()` son final en `LivingThing` no se pueden sobre-escribir, igual que en `Tree`.

2)

```
package domain;  
import java.awt.Color;
```

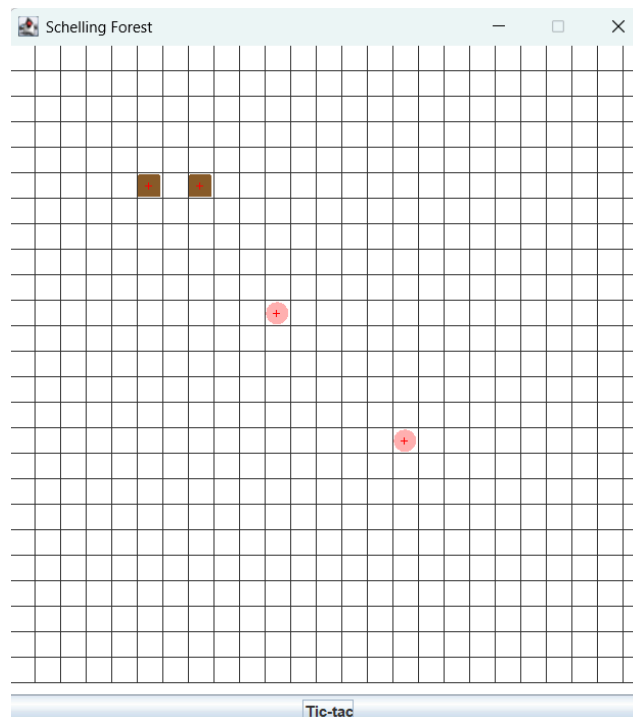
```
public class Squirrel extends LivingThing implements Thing {  
  
    private Forest forest;  
    protected int row, column;  
    protected Color color;  
  
    public Squirrel(Forest forest, int row, int column) {  
        this.forest = forest;  
        this.row = row;  
        this.column = column;  
        this.color = new Color(139, 90, 43); // café inicial  
        this.forest.setThing(row, column, this);  
    }  
}
```

El color café se define usando el modelo RGB, donde se asigna un valor alto al rojo, medio al verde y bajo al azul, generando visualmente un tono marrón

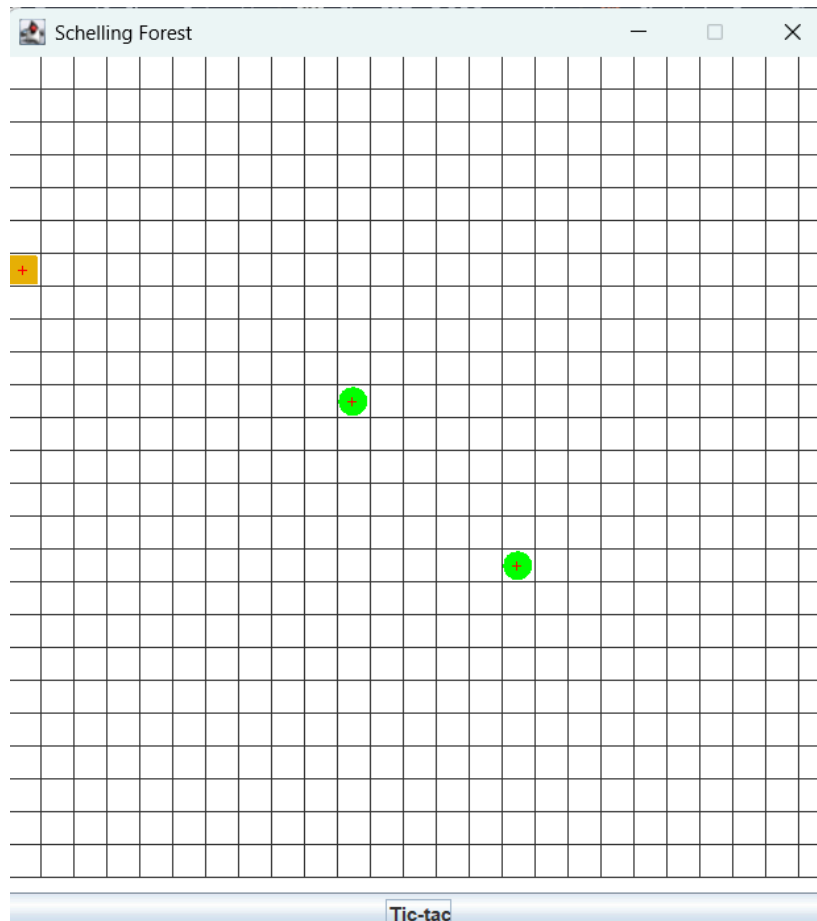


3)

a)



después de 5 tic tacs:



b) Sí, con una observación importante: las posiciones de scrat y sandy después del tic 1 son impredecibles porque `move()` elige una celda adyacente libre aleatoriamente. Lo que sí es determinista y verificable es:

- years y energy avanzan exactamente 1 por tic
- El color en tic 5 es exactamente la interpolación a $t=0.5$ entre café y amarillo
- En tic 1 nace una cría en (5,6) porque es la celda vacía entre `scrat(5,5)` y `sandy(5,7)`.
- Ninguna muere antes del tic 10.

C Ciclo 3. Adicionando sombras [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es incluir sombras (sólo vamos a permitir el tipo básico). Ellas (i) son puntos negros que exparten su sombra en toda la fila en la que se encuentran, (ii) se desplazan de sur a norte circularmente

1. Para poder adicionar sombras , (a) ¿debe cambiar en el código de **Forest** en algo? (b) ¿por qué? (c) ¿debe extender **Thing** ? (d) ¿por qué?
2. Diseñen , construyan y prueben esta nueva clase. (Mínimo dos pruebas de unidad)
3. Adicionen dos objetos sombras en **Forest** , llámenlos **thief** y **lass** , (a) ¿Cómo quedarían después de cuatro Tic-tac? Ejecuten el programa y hagan 7 clics en el botón. Capturen una pantalla significativa. (b) ¿Es correcto?

1)

- a) No. Forest no necesita ningún cambio.
- b) Porque Forest almacena **Thing[][]** , cualquier objeto que implemente **Thing** cabe automáticamente en **places[][]**. El método **ticTac()** de **Forest** ya itera sobre toda la cuadrícula llamando **places[r][c].ticTac()** sin importar qué tipo concreto sea. El polimorfismo hace todo el trabajo.
- c) Sí, debe implementar **Thing**.
- d) Porque **places[][]** es de tipo **Thing[][]**. Si **Shadow** no implementa **Thing**, el compilador rechaza **forest.setThing(row, col, this)**. Además **Shadow** no extiende **LivingThing** — no tiene energía ni años, no es un ser vivo. Solo implementa la interfaz directamente, igual que podría hacerlo cualquier objeto del bosque que no sea viviente.

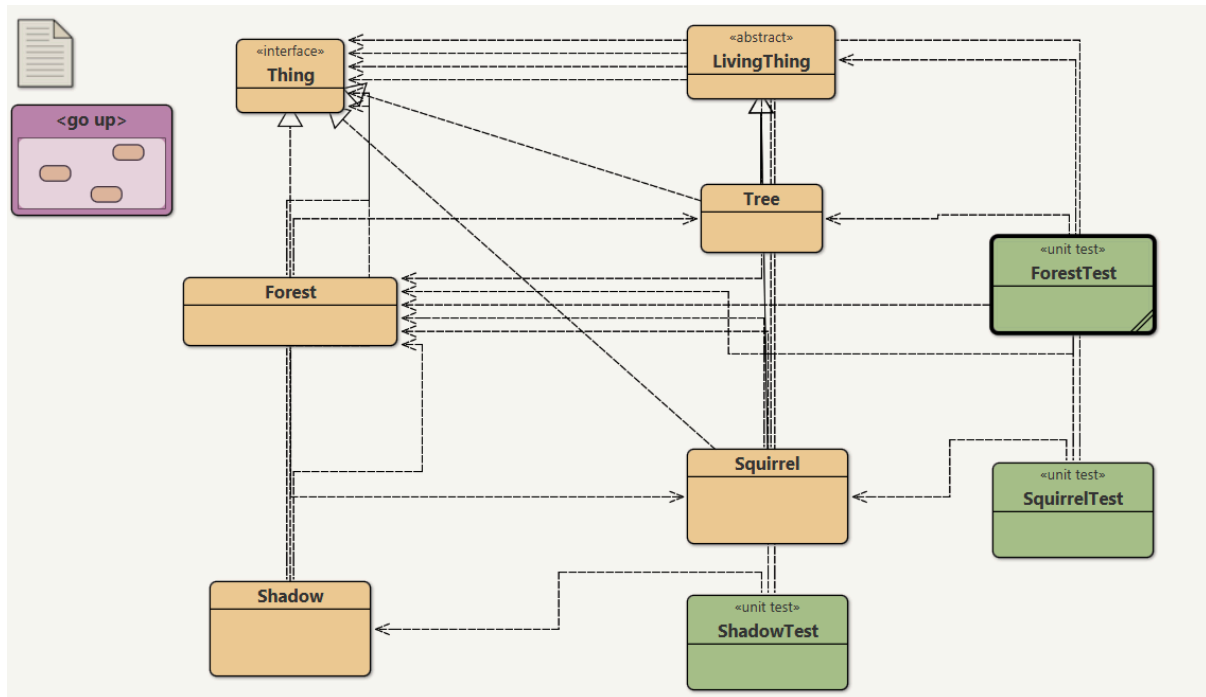
2)

```
package domain;
import java.awt.Color;

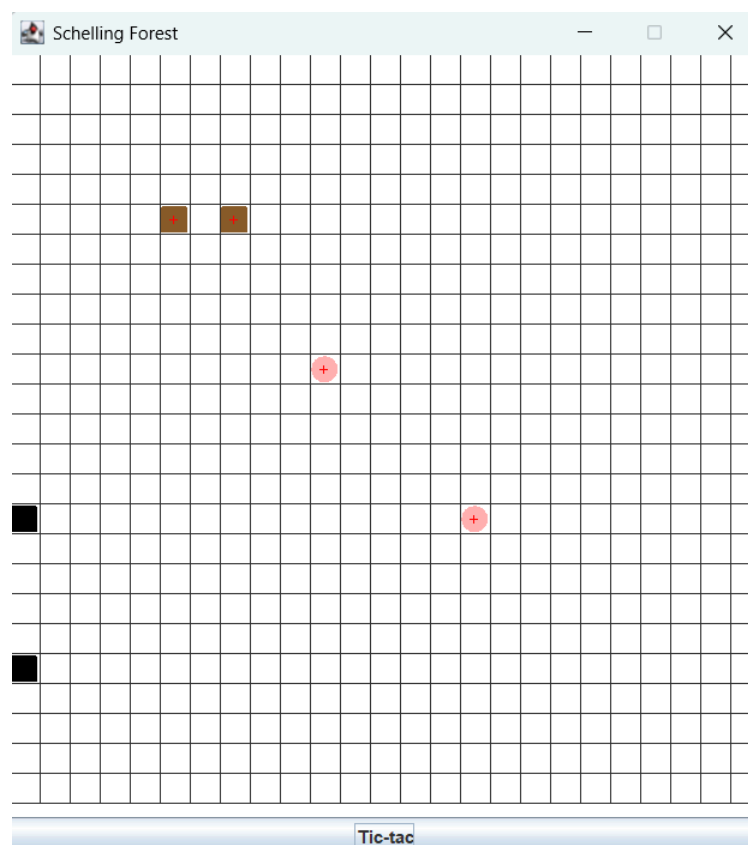
public class Shadow implements Thing {

    private Forest forest;
    private int row;
    private int column;

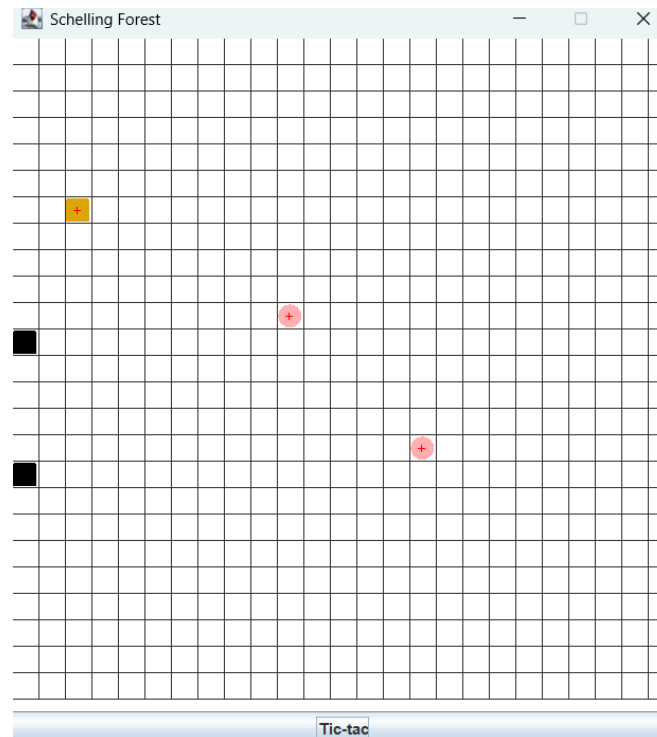
    public Shadow(Forest forest, int row, int column) {
        this.forest = forest;
        this.row = row;
        this.column = column;
        forest.setThing(row, column, this);
    }
}
```



3) Inicial

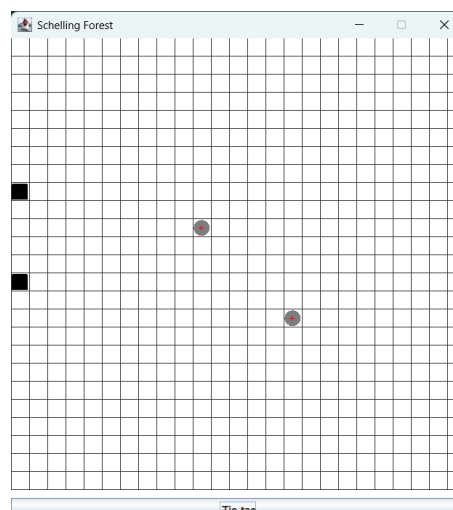


Después de 4 tic tacs:



- Sí. Tras 4 tic-tacs el resultado es exactamente el esperado:
 - thief avanza de fila 20 $\rightarrow$ 16 (resta 1 por tic).
 - lass avanza de fila 15 $\rightarrow$ 11 (resta 1 por tic).
 - Cada una pinta su fila completa de negro, liberando la anterior.
 - soul en (15,15) sobrevive porque isEmpty retorna false — la sombra nunca sobrescribe celdas ocupadas.
 - El movimiento circular no aplica aún porque ninguna llegó a fila 0.
 - Lo único que confirma la ejecución real que el simulador no puede predecir es la posición exacta de scrat y sandy, que se mueven aleatoriamente.

Despues de 7:



D Ciclo 4. Proponiendo y diseñando : Nuevo árbol [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir en un nuevo tipo de `Tree` .

1. Propongan, describan e implementen el nuevo tipo de `Tree` . (Mínimo dos pruebas de unidad)
2. Considerando una pareja de ellos con el apellido de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

1) RainTree (Árbol de lluvia) es un árbol especial que:

- Tiene color azulado que se intensifica con la lluvia — oscurece progresivamente cada 2 tics.
- Riega sus vecinos: cada tic, si hay una celda vacía adyacente, hace crecer un nuevo Tree básico allí (se reproduce en el bosque).
- Vive el doble que un árbol normal — tiene energía inicial de 200 en lugar de 100.
- Su forma es SQUARE para diferenciarse visualmente del Tree normal (que es ROUND).

2)

```
@Test
public void testLassoYGaitanRieganSinConflicto() {
    new RainTree(forest, 12, 12); // lasso
    new RainTree(forest, 12, 14); // gaitan

    forest.ticTac(); // tic 1
    assertNull("(12,13) debe estar vacía en tic 1",
        forest.getThing(12, 13));

    forest.ticTac(); // tic 2

    // lasso riega al primer vecino libre (norte: 11,12)
    // gaitan riega al primer vecino libre (norte: 11,14)
    boolean lassoRiego =
        forest.getThing(11, 12) instanceof Tree ||
        forest.getThing(12, 13) instanceof Tree ||
        forest.getThing(13, 12) instanceof Tree ||
        forest.getThing(12, 11) instanceof Tree;

    assertTrue("lasso debe haber regado un vecino", lassoRiego);

    // lasso y gaitan siguen vivos
    assertNotNull("lasso debe seguir vivo", forest.getThing(12, 12));
    assertNotNull("gaitan debe seguir vivo", forest.getThing(12, 14));
}
```

La prueba verifica que dos RainTree vecinos riegan correctamente sin interferirse entre sí, y que el bosque es consistente tras el tic.

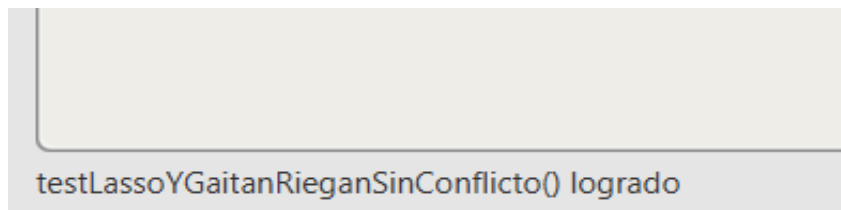
El escenario es: (12,11) (12,12) (12,13) (12,14) (12,15)
 vacía lasso vacía gaitan vacía

Lo que se quiere confirmar es:

En tic 1 ninguno riega , confirma que water() solo actúa en tics pares. La celda (12,13) permanece vacía.

En tic 2 ambos riegan simultáneamente sobre el snapshot ,lasso planta un árbol en su primer vecino libre, gaitan planta un árbol en su primer vecino libre. Como Forest.ticTac() trabaja sobre el snapshot del estado anterior, ninguno ve los árboles que el otro acaba de plantar, cada uno actúa de forma independiente sin pisarse.

Lasso y gaitan siguen vivos después de regar — el acto de regar no los afecta ni los elimina.



E Ciclo 5. Proponiendo y diseñando : Nueva cosa [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo de este punto es permitir recibir una nueva clase de Thing (no Tree) en el Forest

1. Propongan, describan e implementen la nueva clase de Thing . (Mínimo dos pruebas de unidad)
2. Considerando un par de ellos con el nombre de ustedes. (a) Piensen en otra prueba significativa y expliquen la intención. (b) Codifiquen la prueba de unidad correspondiente y capturen la pantalla de resultados de ejecución de la prueba. (c) Ejecuten el programa con esa prueba como prueba de aceptación y capturen las pantallas correspondientes.

1) River (Río) es una cosa del bosque que no es un árbol ni un ser vivo. Sus comportamientos:

- Es de color azul que varía de claro a oscuro cíclicamente simulando el flujo del agua.
- Se expande horizontalmente, cada tic intenta ocupar una celda vacía a su derecha.
- Si llega al borde derecho del bosque, reaparece por la izquierda de su fila (circular).
- Su forma es SQUARE.
- No es LivingThing, implementa Thing directamente, igual que Shadow.

b)

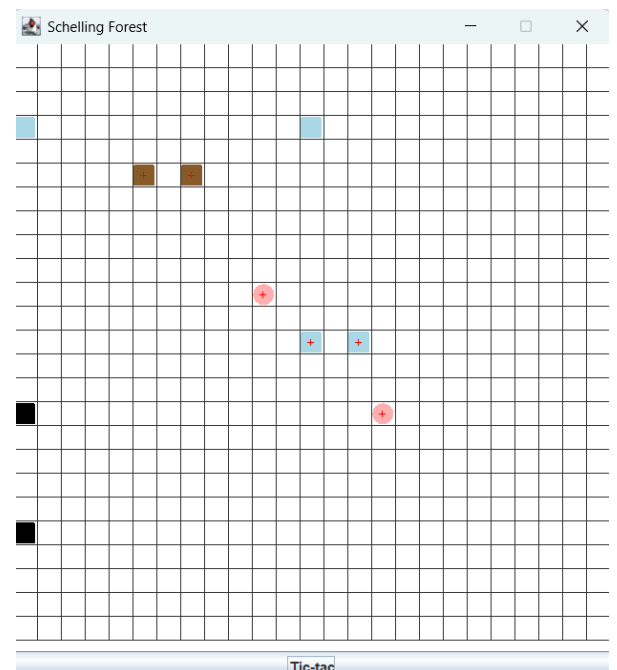
```
@Test
public void testOscarYJuanDiegoNuncaSeSolapan() {
    River oscar      = new River(forest, 3, 0);
    River juanDiego = new River(forest, 3, 12);

    for (int i = 0; i < 20; i++) {
        forest.ticTac();

        // contar cuántos ríos hay en la fila 3
        int count = 0;
        for (int c = 0; c < forest.getSize(); c++) {
            if (forest.getThing(3, c) instanceof River) count++;
        }
        assertEquals("debe haber exactamente 2 ríos en fila 3"
            + " en tic " + (i + 1), 2, count);
    }
}
```

c)

```
public void someThings(){
    new Tree(this, 10, 10);    // beard
    new Tree(this, 15, 15);    // soul
    new Squirrel(this, 5, 5);  // scrat
    new Squirrel(this, 5, 7);  // sandy
    new Shadow(this, 20, 0);   // thief
    new Shadow(this, 15, 0);   // lass
    new RainTree(this, 12, 12); // lasso
    new RainTree(this, 12, 14); // gaitan
    new River(this, 3, 0);     // oscar
    new River(this, 3, 12);    // juanDiego
}
```



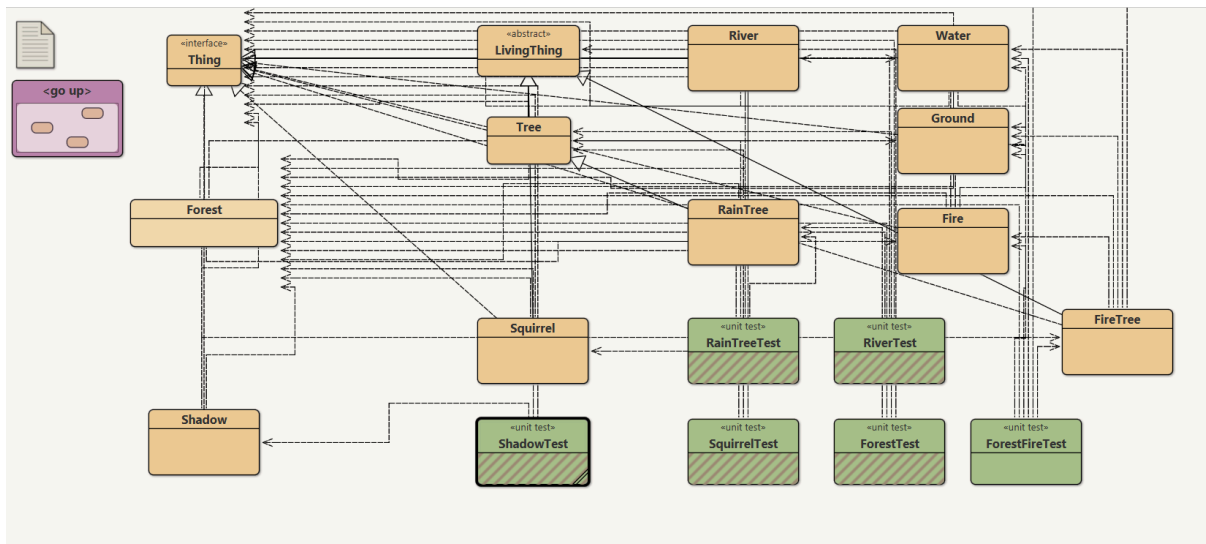
PARTE III. Desarrollando el reto [En lab03.doc forest.astah *.java]

(NO OLVIDE BDD – MDD)

En la zona del bosque dedicada al simulador *Forest Fire* vamos a tener únicamente cuatro elementos: árboles, fuego, agua y tierra. En esa zona no pueden existir celdas vacías. Cada celda tiene ocho vecinos (vecindario de Moore).

- Si un árbol débil queda cerca al fuego pierde el 25 % de su energía y si queda con menos de 10 % de energía se prende, convirtiéndose en fuego.
- Si un árbol queda cerca a una fuente de agua recupera 100 % de su energía. El agua que lo alimentó se agota, convirtiéndose en tierra.
- Si un árbol queda rodeado sólo por tierra genera un nuevo árbol en una de las celdas vecinas.
- Si el fuego queda cerca al agua, se extingue; y dura cinco ciclos, si permanece rodeado únicamente por tierra. Al extinguirse se convierte en tierra.
- Si el agua está rodeada por tierra, intenta moverse hacia el sur. El espacio que deja se convierte en tierra.
- La tierra puede generar fuego con una probabilidad de 10 % y agua con probabilidad de 5 %.

Estos elementos en cada momento deciden la acción a realizar, la primera posible siguiendo el orden en que se presentan las reglas, y la realizan inmediatamente.

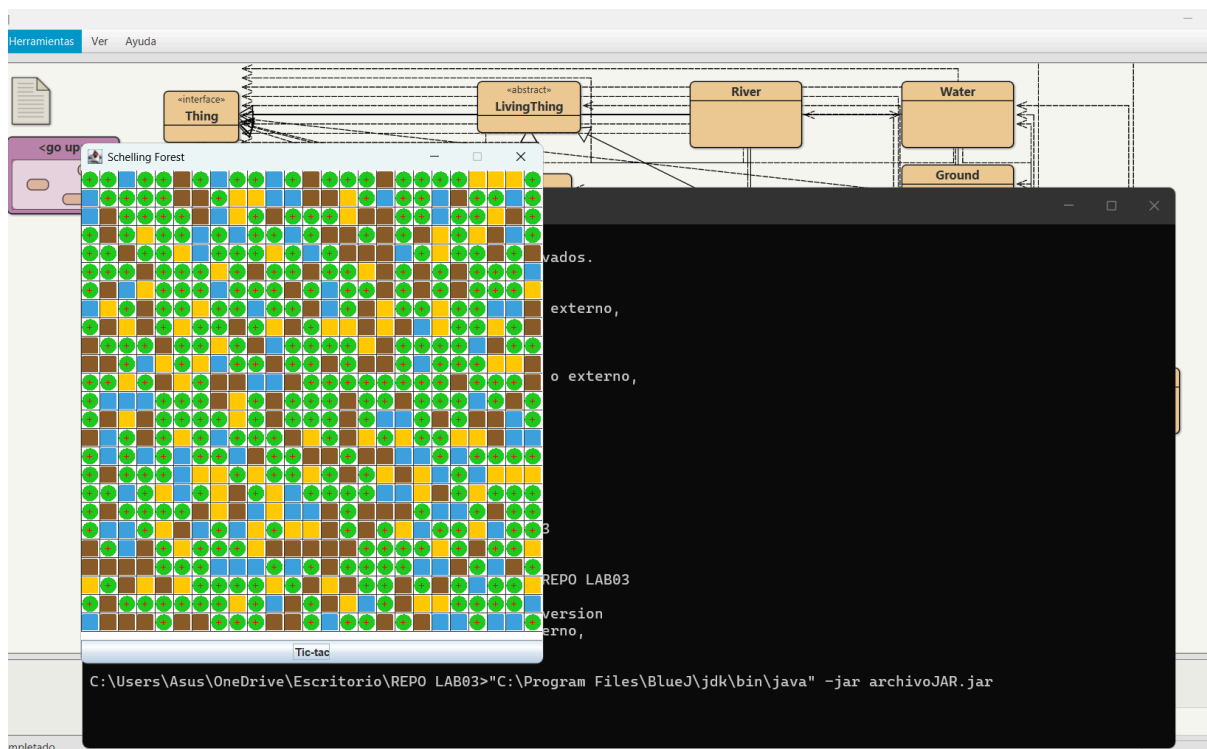


PARTE IV. Empaquetando el proyecto [En lab03.doc forest.jar]

En un **JAR** (Java ARchive) se pueden empaquetar múltiples clases y archivos para distribuir y ejecutar artefactos software en un solo archivo comprimido.

1. Revise las opciones de **BlueJ** para empaquetar su programa en un archivo **.jar**. Genere el archivo correspondiente. (a) Incluya una pantalla con evidencia de la existencia del archivo.
2. Consulte el comando java para ejecutar un archivo **.jar**. ejecútenlo (a) ¿qué pasa?
3. (a) ¿Qué ventajas tiene esta forma de entregar los proyectos? Explique claramente.

Lab03	21/03/2026 9:27 p. m.	Carpeta de archivos
archivoJAR.jar	21/03/2026 11:25 p. m.	Archivo JAR



c)

Un solo archivo — todo el proyecto empaquetado en un .jar. No faltan clases ni recursos al momento de entregar.

No requiere BlueJ — el evaluador solo necesita Java instalado para ejecutarlo con `java -jar archivoJAR.jar`.

Multiplataforma — el mismo archivo corre en Windows, Mac y Linux sin recompilar.

Distribución simple — se envía fácilmente por correo, campus virtual o repositorio como cualquier archivo adjunto.

Protección básica — contiene .class compilados, no el código fuente .java directamente legible.

Retrospectiva

1. Tiempo invertido

10 horas cada uno — 20 horas en total como equipo.

2. Estado actual del laboratorio

El laboratorio está completo en un 90%. Se implementaron exitosamente todos los ciclos: árboles, ardillas, sombras, RainTree, River y el simulador Forest Fire. Queda pendiente pulir detalles menores como el null-check en los bordes del bosque y el método restore() en LivingThing. El JAR fue generado y ejecutado correctamente.

3. Práctica XP más útil

Testing — escribir las pruebas primero (TDD). Fue la más útil porque permitió detectar errores de diseño antes de implementar. Por ejemplo, el conflicto entre lasso y gaitan en RainTree solo se descubrió gracias al test — sin él el error hubiera pasado desapercibido. Las pruebas también forzaron a pensar en los casos límite como celdas nulas y bordes del bosque.

4. Mayor logro

Implementar el simulador Forest Fire completo con sus cuatro elementos y todas las reglas de interacción. Fue el reto más complejo porque requirió coordinar múltiples clases que se transforman entre sí manteniendo la invariante de no tener celdas vacías, además de corregir el Forest.ticTac() con el snapshot para evitar que objetos recién nacidos actuaran en el mismo tic.

5. Mayor problema técnico

El error más difícil fue el conflicto en Forest.ticTac() donde los objetos creados durante un tic actuaban en ese mismo ciclo, causando comportamientos impredecibles en RainTree y el simulador. Se resolvió implementando un snapshot de places[][] al inicio de cada tic, de modo que solo los objetos preexistentes actúan en ese ciclo.

6. Trabajo en equipo

Hicimos bien: la división de responsabilidades por ciclo — cada uno implementaba una clase y el otro escribía las pruebas, lo que redujo errores y aceleró el desarrollo.

Nos comprometemos a: comenzar las pruebas antes de implementar en lugar de escribirlas después, y revisar el diseño en el diagrama antes de tocar el código para evitar refactorizaciones tardías.

7. Referencias

- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.
- Beck, K. (2002). *Test Driven Development: By Example*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle. (2024). *Java SE 17 Documentation*.
<https://docs.oracle.com/en/java/javase/17/>